

Assignment-B-Group25: Golden-shovels Refactory

1. Team Information

Full names	Matriculation numbers	Contact number	Email address
毕雅琳	2252092	18560811136	2574561859@qq.com
付柏瑞	2252426	18041302075	2252426@tongji.edu.cn
付宝莹	2251654	19145163653	987596476@qq.com
罗诗雨	2251926	19979368244	3041739348@qq.com
文彧婕	2252954	19934575906	2429638087@qq.com

2. A Description Of Our Project

2.1 Project Background

Golden-shovels is a game project based on the Cocos2d-x engine, which aims to provide players with an entertainment experience that combines strategy and adventure. The project was chosen based on the current needs of the game market for innovative strategy games, combined with the interests and strengths of the team members in game development. The project will use the Cocos2d-x 3.17 engine, Photoshop 2022 for image design, csm-3d for 3D modeling, VS 2022 for development, and GitHub for version control and team collaboration with SourceTree. The target platform is Windows, with the possibility of expanding to other platforms in the future.

The project is a project of our programming paradigm, based on which we modify software design patterns.

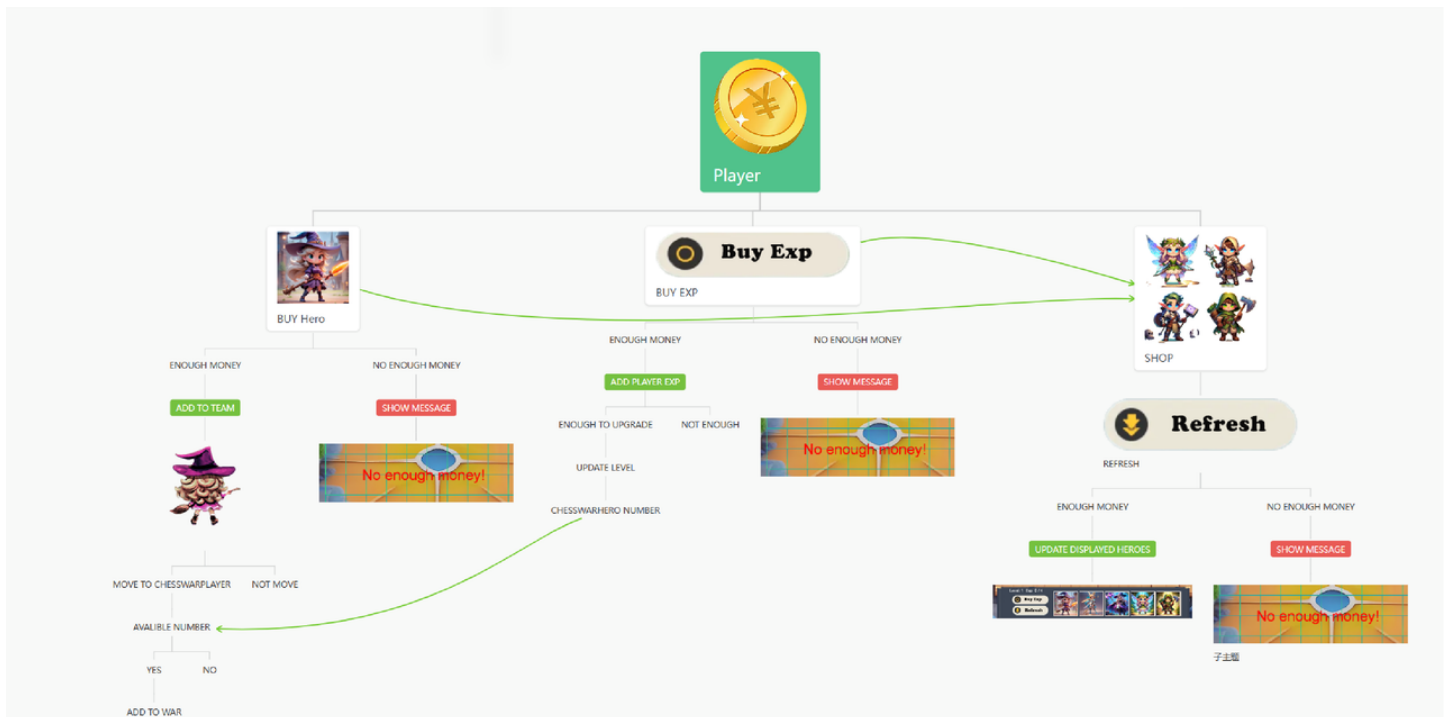


2.2 Main Functions

1. There are initial interface and setup interface, 1v1 map
2. Support setting function, sliding adjustment and opening and closing of background music
3. Support the purchase of multiple types of cards, including fighters, helpers, Mages, and shooters
4. Support card pool and refresh store cards
5. Support card star raising function

6. Support small hero right click movement and health display
7. The cards on the field support red and blue blood bars, which can release skills when the blue bar is full and carry out the whole team's blood return
8. Support practice mode (i.e. single player mode) where players can play against AI players
9. Support two people online, create and join rooms
10. Realized the fight sound effect, success and failure sound effect, key sound effect.
11. At the end of the round, small heroes will be deducted according to the number of remaining heroes, increasing player experience and gold
12. Support player experience upgrades, related to the number of heroes that can play

2.3 Project Design



3. Refactoring Work

3.1 Factory Method (Creational Pattern)

3.1.1 Problem

In the original implementation, the Hero class took on too much responsibility, handling multiple aspects of hero management including initialization, combat, state management, and attribute systems. This design leads to several core problems:

1. **High coupling:** The creation process was particularly problematic. With no standardized creation pattern, heroes could be instantiated in multiple ways, leading to potential initialization errors and inconsistent states. The direct dependency on the Information system for hero data meant that any changes to the data structure required modifications to the Hero class itself. This tight coupling made it nearly impossible to test components in isolation or mock dependencies for unit testing.
2. **Poor flexibility:** The system's inflexibility became apparent when trying to add new hero types or modify existing behaviors. The hard-coded nature of hero creation and initialization meant that extending the system required modifying existing code, violating the Open-Closed Principle. This made it risky to add new features or modify existing ones, as changes could have unforeseen consequences throughout the codebase.

3.1.2 Solution

These above issues collectively created a system that was not only difficult to maintain but also resistant to change and prone to errors. The Factory Method pattern addresses these core problems by providing a standardized creation process, a flexible structure for adding new hero types without modifying existing code, and making client code unaware of the specific class being instantiated.

Implementation steps:

1. Make all products(`ConcreteHero`) follow the same interface(`IHero`), which declares methods that make sense in every product(e.g. `TaricHero` , `KaterinaHero`).
2. Add an empty factory method inside the creator class(`HeroCreator`), with the return type matching the interface(`IHero`).
3. Find all references to product constructors(e.g. `Hero* hero = Hero::create("hero_image.png");`), and replace them with calls to the factory method, while extracting the product creation code into the factory method.
4. Create a set of creator subclasses for each type of product(e.g. `TaricCreator` , `KaterinaCreator`), and override the factory method in the subclasses
5. If there are too many product types and it does not make sense to create subclasses for all of them, you can reuse the control parameter (`heroTag`) from the base class in subclasses.
6. If the base factory method has become empty, make it abstract; if there is something left, make it a default behavior of the method

3.1.3 Code refactoring

The code we refactored is hero.cpp and hero.h files, which mainly include the Hero class. In order to use Factory Method, we also added HeroCreatorRegistry.h, HeroCreator.h,

ConcreteCreator.h, IHero.h and ConcreteHero.h. Here are some of the important refactoring changes:



Before refactoring:

- Creation and initialization logic were exposed to the external parts, which was problematic and unacceptable.
- Hard to maintain and extend, which means we have to change existing codes if we want to add more heroes or more performing logic.
- No standardized creation process, including being prone to initialization errors and hard to ensure complete initialization

This is the original `Hero` class.

```
1 // Base Hero(product)
2 class Hero :public cocos2d::Sprite, public IHero
3 {
4 public:
5     static Hero* create(const std::string& filename);
6     Hero();
7     //other functions...
8 private:
9     Sprite* star;
10    int id;
11    int level;
12    cocos2d::Sprite* starSprite;
13    std::string heroName;
14    heroInformation* baseInfo;
15    Vec2 heroPos;
16    showHPMP* heroHP;
17    showHPMP* heroMP;
18    std::unique_ptr<Attack> heroAttack;
19    std::unique_ptr<IAttackBehavior> attackBehavior;
20    Hero* atkTarget;
21    //other fields...
22 };
```

And this is the original code to create a `Hero` type instance. We can tell that there is no standardized creation process, leading to the chance to cause initialization errors. Moreover, we have to go through the whole initialization process if we want to create a certain type of hero. Also, it's very hard to add new hero and new gaming logic, because we have to change the existing codes.


```

1 // Multiple ways to create heroes
2 Hero* hero1 = Hero::create("hero.png");
3 hero1->SetHeroDataByTag(1, pos);
4
5 Hero* hero2 = new Hero();
6 hero2->SetBaseInfo(info, "Taric", pos);
7 hero2->HPInit();

```

☀ After refactoring:

- Separating creation from use
- Standardizing the creation process
- Encapsulating initialization logic
- Making the system more flexible and testable

We add `IHero` as the interface of all types of concrete heroes, which declares methods that make sense in every hero.

```

1 // Hero(product) Interface
2 class IHero {
3 public:
4     virtual ~IHero() = default;
5     // basic attributes
6     virtual void initAttributes() = 0;
7     virtual void updateAttributes() = 0;
8     virtual int getId() const = 0;
9     virtual void setId(int id) = 0;
10    virtual int getLevel() const = 0;
11    virtual void setLevel(int level) = 0;
12    virtual std::string getName() const = 0;
13    // battle related
14    virtual void performAttack() = 0;
15    virtual void performMove() = 0;
16    virtual void takeDamage(float damage) = 0;
17    virtual double getAttackPower() const = 0;
18    virtual void startBattle(IHero* target) = 0;
19    // state related
20    virtual bool isDead() const = 0;
21    virtual bool isOnBattle() const = 0;
22    virtual void setOnBattle(bool on) = 0;
23    // position related

```

```

24     virtual void setPosition(Vec2 pos) = 0;
25     virtual Vec2 getPosition() const = 0;
26     virtual Vec2 getTargetPosition() const = 0;
27 };

```

Considering every hero have many common fields and logic and only varies in some behaviors, we assume the original `Hero` class as the base class of all concrete heroes, in order not to destroy the existing codes. Also, it should implement the `IHero` interface.

```

1  // Base Hero(product)
2  class Hero :public cocos2d::Sprite, public IHero
3  {
4  public:
5      //same as before
6
7      // Implement IHero interface
8      void initAttributes() override;
9      void updateAttributes() override;
10     int getId() const override { return id; }
11     void setId(int id) override { this->id = id; }
12     int getLevel() const override { return level; }
13     void setLevel(int level) override;
14     std::string getName() const override { return heroName; }
15     void performAttack() override;
16     void performMove() override;
17     void takeDamage(float damage) override { ChangeHP(-damage); }
18     double getAttackPower() const override { return GetAtk(); }
19     void startBattle(IHero* target) override;
20     bool isDead() const override { return isdead; }
21     bool isOnBattle() const override { return ison; }
22     void setOnBattle(bool on) override { SetOn(on ? 1 : 0); }
23     void setPosition(Vec2 pos) override;
24     Vec2 getPosition() const override { return GetHeroPos(); }
25     Vec2 getTargetPosition() const override { return GetTargetPos(); }
26 private:
27     //same as before
28 };

```

```

1  // Concrete Hero(product)
2  class TaricHero : public Hero {
3  public:
4      static TaricHero* create() {
5          // get data from Information class

```

```

6      std::string imagePath = Information::getHeroPathByTag(1);
7      TaricHero* hero = new TaricHero();
8
9      if (hero && hero->Hero::create(imagePath)) {
10         hero->autorelease();
11
12         heroInformation* info = Information::GetHeroBaseInfo(1);
13         if (!info) {
14             CC_SAFE_DELETE(hero);
15             return nullptr;
16         }
17         hero->setId(1);
18         hero->SetBaseInfo(info, Information::getNameByTag(1), Vec2::ZERO);
19         hero->HPInit();
20
21         return hero;
22     }
23     CC_SAFE_DELETE(hero);
24     return nullptr;
25 }
26 };
27
28 // other heroes...

```

Next, we build `HeroCreator` class as an abstract factory and also its creator subclasses for each type of hero. More importantly, we use a control parameter `heroTag` in `HeroCreatorRegistry` class to decide which hero to create based on its tag.

```

1 // Hero Creator(factory)
2 class HeroCreator {
3 public:
4     virtual ~HeroCreator() = default;
5
6     virtual IHero* createHero(Vec2 pos) = 0;
7 };

```

```

1 // Concrete hero creator(factory)
2 class TaricCreator : public HeroCreator {
3 public:
4     IHero* createHero(Vec2 pos) override {
5         TaricHero* hero = TaricHero::create();
6         if (!hero) return nullptr;
7

```



```

8      hero->setPosition(pos);
9      return hero;
10   }
11 };
12 //other heroes...

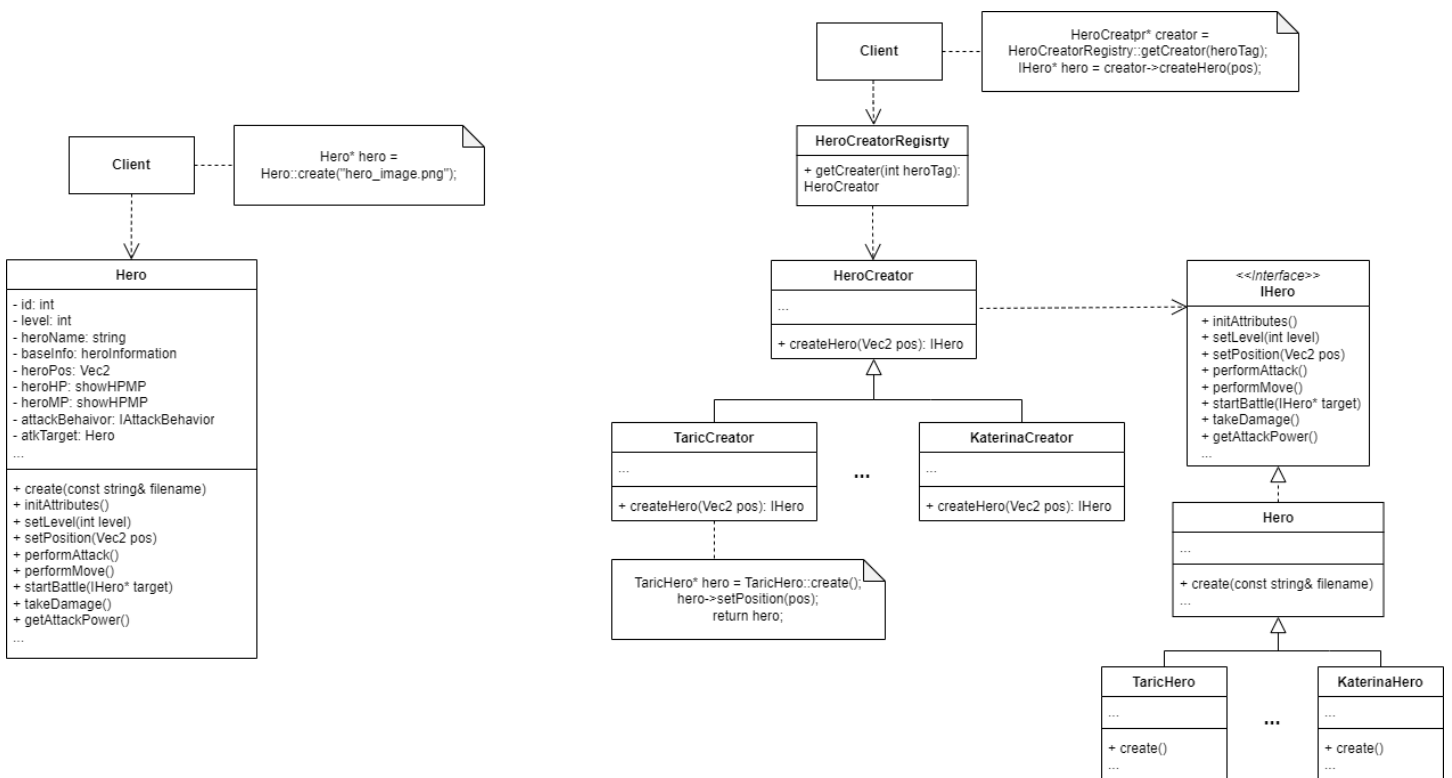
```

```

1 // Factory registry
2 class HeroCreatorRegistry {
3 public:
4     static HeroCreator* getCreator(int heroTag) {
5         switch (heroTag) {
6             case 1: return new TaricCreator();
7             case 2: return new AliCreator();
8             // ... other hero creators
9             default: return nullptr;
10        }
11    }
12 };

```

3.1.4 Class diagrams



Left figure: Hero class diagram before refactoring

Right figure: Hero class diagram after refactoring with Factory Method

Before Refactoring Class Diagram Explanation:

There's only one class that contains all the logic about heroes. There's no standardized creation process, leading to being prone to initialization errors and hard to ensure complete initialization.

After Refactoring Class Diagram Explanation:

This structure follows the Factory Method Pattern, providing a clear hierarchy and separation of concerns in the hero creation system

1. IHero (Product Interface):

- Defines the interface that all heroes must implement
- Contains basic attributes and behavior methods

2. Hero (Abstract Product):

- Implements the IHero interface as a base class
- Provides common hero functionality implementation

3. ConcreteHeroes (Concrete Products):

- Specific hero classes (Taric, Ali, etc.)
- Inherit from the Hero class

4. HeroCreator (Creator):

- Abstract factory class
- Declares factory method createHero()

5. ConcreteCreators (Concrete Creators):

- Specific hero creator classes
- Implement createHero() method

6. HeroCreatorRegistry (Factory Registry):

- Manages all hero creators
- Provides method to get corresponding creator by hero ID

7. Relationship Description:

- `Hero` implements `IHero` interface
- Concrete hero classes inherit from `Hero`
- Concrete creators inherit from `HeroCreator`
- Creators create corresponding heroes
- Registry manages all creators

3.1.5 The benefits

1. Better Structure:

- a. **Single Responsibility for Each Class:** Each class now has a clear and singular responsibility. The `HeroCreator` class (or its subclasses) is solely responsible for creating heroes, while the individual hero classes (like `TaricHero`, `KaterinaHero`, etc.) are focused on their specific behavior and properties. This separation ensures that classes don't take on unnecessary responsibilities and adhere to the **Single Responsibility Principle** (SRP).
- b. **Standardized Creation Process:** By using a factory method, the process of creating heroes is standardized across the entire system. This leads to a uniform way of instantiating hero objects, regardless of their type. Standardization is crucial in larger systems, ensuring that object creation is consistent and centralized, making the codebase easier to maintain and understand.
- c. **Simplified Dependencies:** With the Factory Method in place, the dependencies between classes are simplified. Hero instantiation is abstracted away from the client code, so clients (other classes using the heroes) no longer need to know the concrete classes involved. Instead, they just interact with the interface (`IHero`). This reduces the coupling between the classes and simplifies the dependencies in the system.
- d. **Enforced Complete Initialization:** The Factory Method ensures that objects are created in a valid and consistent state. You can ensure that all the necessary properties (e.g., hero attributes, attack power, defense stats) are correctly initialized before they are used. This avoids the risk of forgetting to set up critical fields or calling uninitialized methods on a new hero object.

2. Enhanced Extensibility:

- a. **Easy to Add New Heroes:** The Factory Method makes it simple to add new hero types. To add a new hero (e.g., `NewHero`), all you need to do is create a new subclass of `HeroCreator` and implement the `createHero` method to return an instance of the new hero. The rest of the system remains unchanged, ensuring that adding new hero types is straightforward and does not require modifying existing code.
- b. **No Modification to Existing Code:** A key advantage of the Factory Method is that you can introduce new hero classes without modifying existing classes. This adheres to the **Open/Closed Principle** (OCP), which states that software should be open for extension but closed for modification. By using a factory, new heroes can be integrated without altering the code that uses or depends on the hero objects, thereby reducing the risk of introducing bugs.

- c. **Clear Extension Points:** The Factory Method clearly defines where and how new hero types should be introduced. The factory method serves as an extension point, meaning that when you want to create a new type of hero, you know exactly where to place the code for hero creation. This clear structure enables smoother collaboration and easier future updates or changes to the system, as it prevents the need for altering core logic.
-

3.2 Facade (Structural Pattern)

3.2.1 Problem

In the original code, multiple subsystems (Battle, Player, Hero, etc.) are directly exposed to external modules (such as MapScene), which brings the following problems:

1. **High coupling:** External modules need to directly call the interfaces of multiple subsystems, and the internal details of each subsystem must be understood. For example, MapScene requires direct manipulation of Battle, Player, and Hero, resulting in high coupling between these classes that are difficult to scale and maintain.
2. **High complexity:** External modules need to manage the initialization and interaction processes of multiple subsystems by themselves, and the code logic is lengthy and complex, and the external modules assume too much responsibility, reducing the readability and reusability of the code.
3. **Difficult to scale:** Once the logic of the subsystem changes, the external modules also need to be modified synchronously. In this case, any expansion requirements, such as adding new battle modes or modifying hero behaviors, would need to be modified in multiple places, increasing development costs and the risk of errors.

3.2.2 Solution

In order to solve the above problems, the Facade Pattern is used for refactoring. The Facade Pattern is a structural pattern, and a BattleSystemFacade class is introduced as a unified interface for all combat-related subsystems, exposing simple operation methods to the outside and hiding the complex interaction logic inside.

Implementation steps:

1. **Encapsulate complex subsystem interactions:** BattleSystemFacade encapsulates the creation, initialization, management, and destruction of Battle, and external modules can complete the battle process by calling interfaces such as startNewBattle.
2. **Hide internal implementation details:** Instead of directly accessing Battle, Player, and Hero methods, external modules operate these subsystems indirectly through the interface provided by BattleSystemFacade.

3. **Provides high-level interfaces:** BattleSystemFacade provides several high-level operation interfaces (such as startNewBattle, endCurrentBattle, etc.) that mask internal details and simplify the process of calling external modules.

3.2.3 Code refactoring

The code we rewrote is Battle.cpp, Battle.h file, and mainly includes the Battle class. And we added BattleSystemFacade.cpp, BattleSystemFacade.h code, defined and implemented the Facade class (BattleSystemFacade). In the refactored code, we encapsulated the operations related to the battle process of the Battle class in the appearance class, providing a higher-level operation interface and simplifying the call process. We've also rewritten some of the original non-standard function definitions in the Battle class to simplify the implementation logic.

1. Changes to the definition of the Battle class:



Before:

A large number of methods are exposed for complex calls by external interfaces.

```
1 // Battle.h
2 class Battle : public Layer {
3 public:
4     void SetPlayer(Player* my, Player* en);
5     void Start();
6     bool IsEnd();
7     bool IsNext();
8     void SetHeroData();
9     // ... other public methods
10 };
```



After:

Allow the **Facade** to access internal methods and lower the access level of methods.

```
1 // Battle.h
2 class Battle : public Layer {
3     friend class BattleSystemFacade; // key change: add a friend class
4
5 public:
6     CREATE_FUNC(Battle);
7     virtual bool init();
```

```

8
9 protected: // key change: lower the access level of methods
10     void setPlayers(Player* my, Player* en);
11     bool startBattle();
12     void setupHeroes();
13     bool isGameOver();
14     bool isNextRound();
15     bool isInProgress() const;
16
17     // internal methods
18     void setAttackTargets(Hero** atk, Hero** tar, int atkNum, int tarNum);
19     void updateTargets(Hero** atk, Hero** tar, int atkNum, int tarNum);
20     // ...
21
22 private:
23     Hero* self[MaxHero];
24     Hero* enemy[MaxHero];
25     // ... other variable member
26 };

```

Add friendly meta classes and reduce the access level of methods, and through the refactoring of this part of the code, prepare for the subsequent creation of Facade classes to be responsible for the unified management of the battle system.

2. Introduction of BattleSystemFacade:



New Classes:

```

1 // BattleSystemFacade.h
2 class BattleSystemFacade {
3 public:
4     //Obtain the instance in singleton pattern
5     static BattleSystemFacade* getInstance();
6
7     // Core battle flow control
8     bool startNewBattle(Player* player1, Player* player2, Scene* scene);
9     void endCurrentBattle();
10    bool isBattleInProgress() const;
11    bool isReadyForNextRound() const;
12
13    // battle operation
14    void executeAttack(Hero** attackers, Hero** targets, int attackerCount, int
    targetCount);

```



```

15     void updateBattleTargets(Hero** attackers, Hero** targets, int
    attackerCount, int targetCount);
16     void resetBattleField();
17     void healHeroes(Hero** heroes, int count, double amount);
18
19     // status query
20     Player* getFirstPlayer() const;
21     Player* getSecondPlayer() const;
22     void updateBattle(float dt);
23
24 private:
25     Battle* currentBattle{nullptr};
26     // ... Singleton related code
27 };

```

With the introduction of this Façade class, the singleton pattern ensures that there is only one instance of the battle system in the entire game, and provides a global access point that can be called up by external modules at any time.

Its core process control interfaces include 'startNewBattle' (to create and initialize a new battle), 'endCurrentBattle' (to end the current battle and clear resources), 'isBattleInProgress' (to check if the battle is in progress), and 'isReadyForNextRound' (to check whether it can be advanced to the next round), so as to achieve unified management of the entire battle process.

At the same time, the battle operation interface provides a unified battle operation method to the outside world, encapsulates the specific battle implementation details, and enables external callers to operate the battle without caring about internal logic.

In addition, the status query interface is convenient for external systems to obtain the current battle status by calling simple methods, so as to understand the progress of the battle or judge the next operation, which greatly improves the ease of use and maintainability of the system.

3. MapScene uses the battle system through BattleSystemFacade:



Before:

Numerous functions within the **Battle** class are called to create and update a battle system in a complex manner.

```

1 // MapScene.h
2 class MapScene {
3 private:
4     Battle* battlelayer;

```

```

5      // ...
6
7  void MapScene::CreateBattle(float t) {
8      battlelayer = Battle::createLayer();
9      battlelayer->SetPlayer(player, enPlayer);
10     battlelayer->Start();
11     this->addChild(battlelayer);
12 }
13
14 void MapScene::update(float dt) {
15     if (battlelayer->IsNext()) {
16         // End of processing round...
17     }
18
19     if (battlelayer->IsEnd()) {
20         // Deal with the end of the battle...
21     }
22 }

```



After:

Changed the creation/update of the battle system from multiple methods that directly call the Batt class to operate by BattleSystemFacade

```

1  // MapScene.h
2  class MapScene {
3  private:
4      BattleSystemFacade* battleSystem;
5
6  void MapScene::CreateBattle(float t) {
7      battleSystem = BattleSystemFacade::getInstance();
8      battleSystem->startNewBattle(player, enPlayer, this);
9  }
10
11 void MapScene::update(float dt) {
12     if (battleSystem->isReadyForNextRound()) {
13         // End of processing round...
14     }
15
16     if (battleSystem->isBattleInProgress()) {
17         battleSystem->updateBattle(dt);
18     }
19 }

```

By reducing the coupling in the Façade mode, MapScene only needs to know the interface of the BattleSystemFacade, making it easier to create and control the logic of the battle system.

4. Core Architectural Changes:

As a result of the above refactoring, the following core architectural changes have been made.



Before:

MapScene --> Battle (Direct Dependency)



After:

MapScene --> BattleSystemFacade --> Battle (Introducing Facade Pattern)

3.2.4 Class diagrams

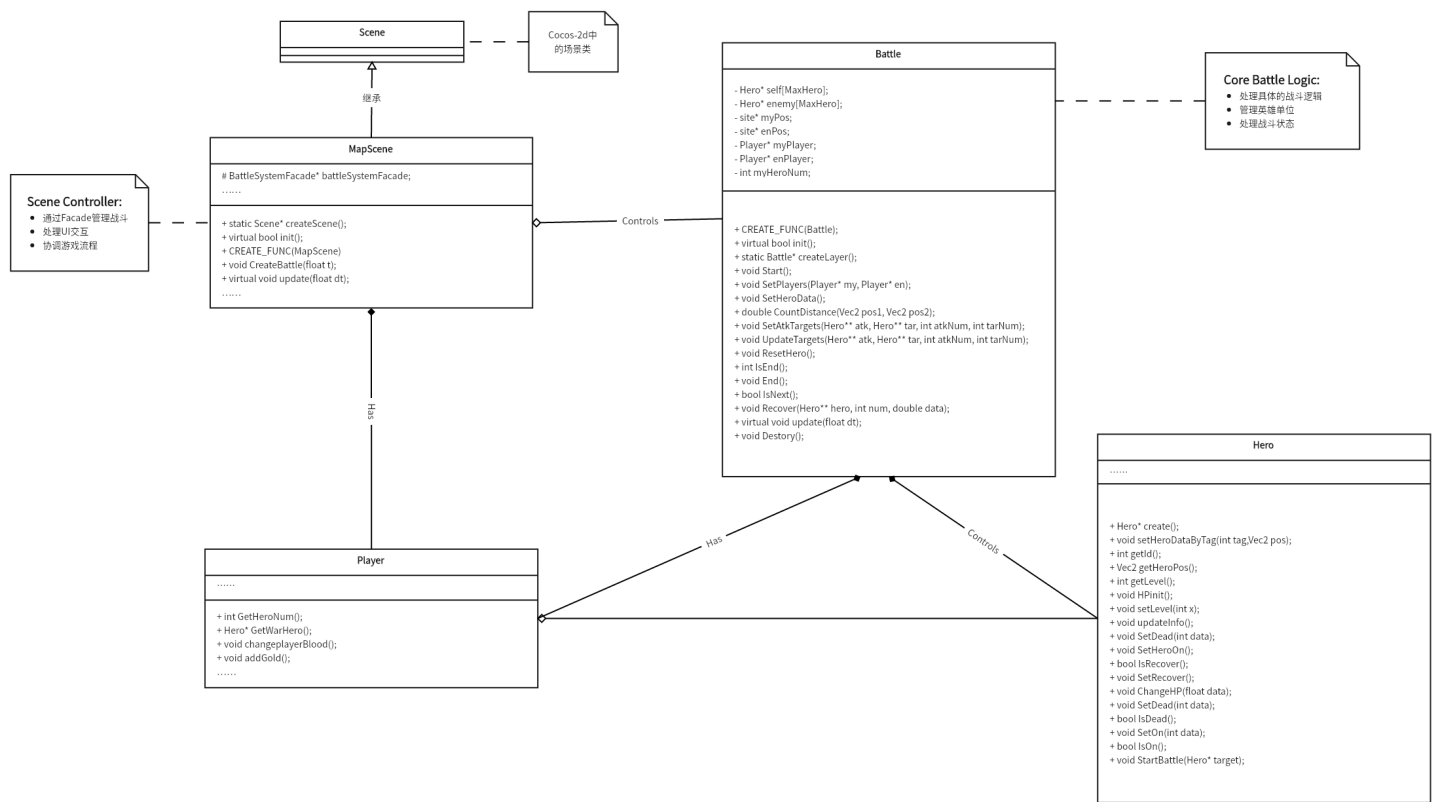


Before:

This is the class diagram of the battle system before the refactory.

Before refactoring, the class relations in the system showed strong coupling:

- MapScene directly holds the instance (battlelayer) of Battle, can directly call all public methods of Battle, and is responsible for the creation, initialization and lifecycle management of Battle. Battle-related status judgment is performed directly through the Battle method
- All methods of the Battle class are exposed, any component can be directly accessed, and other components can interact directly with Battle

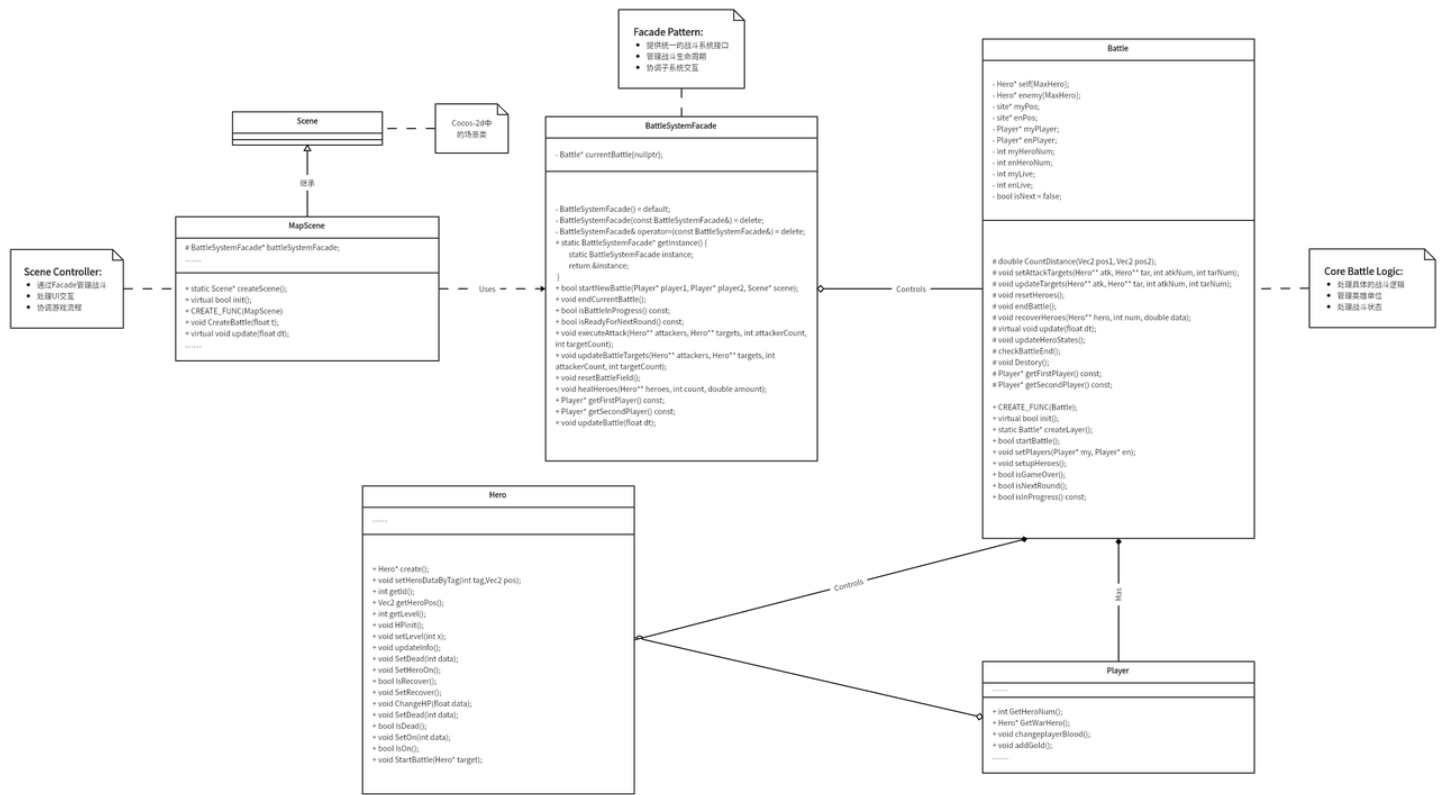


After:

This is the class diagram of the battle system after the refactor.

After refactoring, we introduced a class—BattleSystemFacade.

- MapScene no longer directly holds the Battle instance, only the reference of the BattleSystemFacade, all battle-related operations are carried out through the BattleSystemFacade, and the query of the battle status is completed through the methods provided by the Facade.
- Battle changed most of its methods to protected, making them invisible to external systems, while BattleSystemFacade, as a friendly metaclass of Battle, had access to its internal methods and only allowed access to BattleSystemFacade, making BattleSystemFacade the only class that directly manages Battle.



1. Main Class Description

• Scene:

Scene is the game's base class, representing a game scene that contains everything that is displayed when the game is running. It is the base class provided by **Cocos2d-x**, and **MapScene** inherits from **Scene**.

• MapScene:

- Inherited from **Scene**, it is a specific scene class in the game that manages the map, combat, and player interactions. Contains an instance of **BattleSystemFacade** to control the flow of the battle.
- Interfaces provided:

createScene()	Creates a scene instance
CreateBattle(float t)	Creates a battle via BattleSystemFacade.
update(float dt)	Called every frame to update the scene state in real time.

• BattleSystemFacade:

- **The core Facade class**, which encapsulates all battle-related operations, provides a unified interface to the outside world, and hides the complexity of internal subsystems.
- Main interfaces:

startNewBattle(Player* player1, Player* player2, Scene* scene)	Creates and initializes a new battle.
endCurrentBattle()	Ends the current battle and clears resources.
isBattleInProgress()	Checks if the battle is in progress.
resetBattleField()	Resets the combat state.
Methods such as healHeroes(), executeAttack(), etc.	provide specific combat operations.

- **Battle:**

- **The core combat logic class** is responsible for handling the specific combat process, including the initialization and status update of players and heroes.
- Included attributes:
 - *Hero* self[] and Hero* enemy[]*: Store your own heroes and enemy heroes, respectively.
 - *Player* myPlayer and Player* enPlayer*: Represents friendly and enemy players.
- Main interfaces:

setPlayers(Player* my, Player* en)	Sets the players to participate in the battle.
setupHeroes()	Initializes hero information.
startBattle() and endBattle()	Start and end the battle.
update(float dt)	Updates the battle status in real time.

2. Relationships between classes

- **MapScene uses BattleSystemFacade:** MapScene controls the flow of combat through the interface provided by the BattleSystemFacade, and does not directly interact with Battle, Player, and Hero.
- **BattleSystemFacade controls Battle:** BattleSystemFacade maintains an instance of Battle internally and operates on Battle through higher-level interfaces such as startNewBattle and endCurrentBattle.
- **Battle manages Player and Hero:** Battle holds references to Player and Hero, and is responsible for managing the status, operations, and battle logic of players and heroes.

- **Player manages Hero:** The Player manages the heroes it owns, providing an interface to obtain or manipulate the heroes' attributes.

3. Application of Facade Pattern

- As an appearance class, **BattleSystemFacade encapsulates** the interaction of *Battle*, *Player*, *Hero* and *other subsystems*, and provides simple interfaces to enable external callers (such as MapScene) to complete complex combat operations by interacting with Facade.
- This design **reduces the coupling** between external modules and subsystems, and improves the maintainability and extensibility of the code.

3.2.5 The benefits

1. Reduced coupling:

- External modules, such as MapScene, no longer interact directly with multiple subsystems such as Battle, Player, Hero, etc., and only interact with BattleSystemFacade.
- This low-coupling design allows the subsystem to be modified independently without affecting external modules, improving code maintainability and scalability.

2. Improve code reusability:

- BattleSystemFacade encapsulates the initialization and management logic of battles into high-level interfaces that can be called multiple times by external modules, avoiding duplicate code.
- If you need to add a different battle mode in the future, you can simply extend the method of BattleSystemFacade, and the external module does not need to be modified.

3. Simplify the external call logic:

- The external module only needs to call the high-level interfaces of BattleSystemFacade, such as startNewBattle, endCurrentBattle, etc., without caring about the internal details of the subsystem, and the call process becomes simple and clear.

4. Enhance scalability:

- Because BattleSystemFacade hides internal implementation details, it is possible to add new subsystems or extend existing subsystems inside Facade in the future without affecting external modules. For example, you can add an AIPlayer subsystem for automated matchmaking.

3.3 Bridge Pattern (Structural Pattern)

3.3.1 Problem

The original Attack system suffered from several design issues that made it inflexible and hard to maintain. The `Attack` class was tightly coupled with both movement logic and hit detection, handling everything from position updates to damage calculations. This monolithic approach created several problems:

1. The attack behavior was hardcoded within the Attack class, making it impossible to change attack patterns without modifying the core class.
2. Different heroes needed different attack behaviors, leading to complex conditional logic or duplicate code.
3. Movement patterns and hit detection were tightly coupled, making it difficult to modify one without affecting the other.
4. Adding new attack types required modifying existing code, violating the Open-Closed Principle.
5. Testing different attack behaviors was difficult due to the tight coupling.

3.3.2 Solution

In order to address the above issues, we decided to use Bridge Pattern which transformed a rigid, monolithic system into a flexible, maintainable one that can easily accommodate new attack types and modifications to existing behaviors.

Implementation steps:

1. Identify the orthogonal dimensions in your classes: First dimension is attack display and lifecycle management. Second dimension: Attack behavior (movement patterns and hit detection)
2. Define operations in the `Attack` class(base class)
3. Declare `IAttackBehavior` (implementation interface)
4. Create concrete implementation classes, following the implementation interface
5. Inside the `Attack` class, add a reference field for the implementation type
6. If you have several variants of high-level logic, create refined abstractions for each variant by extending the base abstraction class. (In this case, we don't need refined abstractions because the variations are handled by different behaviors)
7. The client code should pass an implementation object to `Attack`'s constructor

3.3.3 Code refactoring

The code we refactored is `attack.cpp` and `attack.h` files, which mainly include the attack class. In order to use Bridge Pattern, we also added `IAttackBehavior.h` and `AttackBehavior.h`. Here are some of the important refactoring changes:



Before refactoring:

- Different attack methods were implemented by inheriting from the `Attack` class.
- To add a new attack method, a new subclass of `Attack` was required, necessitating modifications to the existing code.
- The behavior of an attack object was fixed and could not be changed dynamically.
- The display logic and behavior logic were mixed together in the same class.

This is the original `Attack` class.

```
1 class Attack : public cocos2d::Sprite
2 {
3 public:
4     CREATE_FUNC(Attack);
5     virtual bool init();
6     //void SetTargetPos(Vec2 data);
7     void SetPosition(Vec2 pos, Size heroSize);
8     //void SetMark();
9     void SetTarget(Hero* atktarget);
10    void GetAttack(double atk);
11    void GetDistance();
12    static Attack* Attack::create(const std::string& filename);
13    void SetName(std::string filename);
14    virtual void update(float dt);
15    void destroy();
16    void timer(float delta);
17 private:
18     Hero* target;
19     Vec2 targetPos;
20     Vec2 direction;
21     double attack;
22     double distance;
23     int isHit = 0;
24     std::string name;
25     };
```



After refactoring:

- Different attack behaviors are achieved by composing different behavior classes.
- You only need to add a new behavior class, without needing to modify the `Attack` class itself.

- Attack behaviors can be dynamically switched at runtime, allowing for more flexibility.
- The `Attack` class is responsible for display and lifecycle management. Meanwhile, the `Behavior` class is responsible for the specific attack behavior.

```

1 // Attack class - Bridge Pattern's Abstraction
2 class Attack : public cocos2d::Sprite {
3 public:
4     // Factory method to create attack behavior based on hero type
5     static std::unique_ptr<IAttackBehavior> createBehaviorForHero(
6         const std::string& heroName, float range) {
7
8         // Create special behaviors for specific heroes
9         if (heroName == "Taric") {
10             return std::make_unique<TaricBehavior>();
11         }
12         // other hero-specific behaviors...
13     }
14
15     // Create attack instance
16     static Attack* create(const std::string& filename, IAttackBehavior*
17         behavior);
18     virtual bool init() override;
19     //...
20 private:
21     IAttackBehavior* behavior; // Attack behavior implementation
22     //...
23 };

```

```

1 // Attack behavior interface - Bridge Pattern's Implementor
2 class IAttackBehavior {
3 public:
4     virtual ~IAttackBehavior() = default;
5     // Handle attack movement
6     virtual void move(Vec2& currentPos, const Vec2& targetPos) = 0;
7     // Check if attack hits target
8     virtual bool checkHit(float distance) = 0;
9     // Get attack's hit range
10    virtual float getHitRange() const = 0;
11    // Get movement speed
12    virtual float getMoveSpeed() const = 0;
13 };

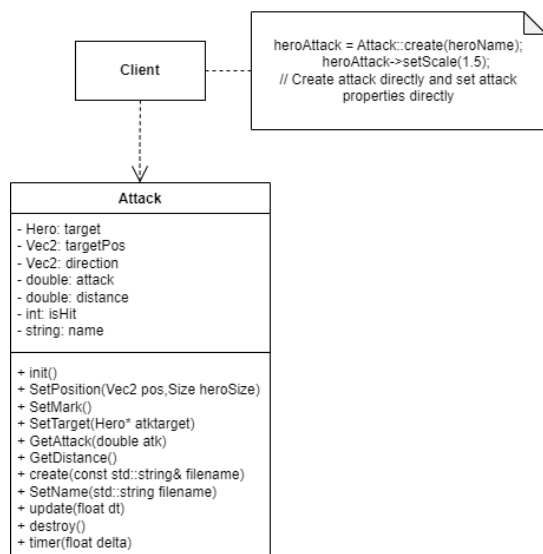
```

```

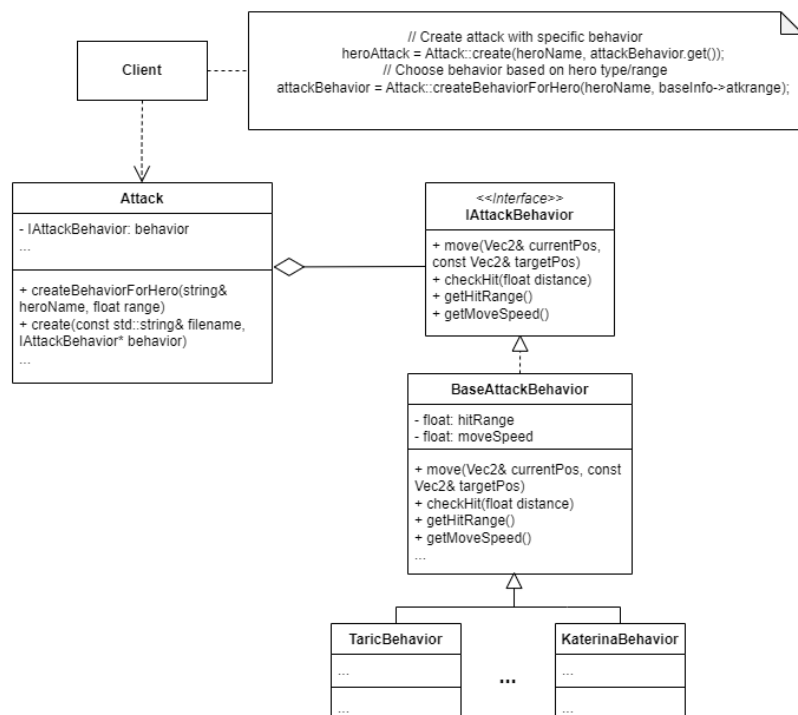
1 // Base attack behavior - Bridge Pattern's Concrete Implementor
2 class BaseAttackBehavior : public IAttackBehavior {
3 protected:
4     float hitRange;    // Range within which attack hits
5     float moveSpeed;   // Movement speed of attack
6 public:
7     BaseAttackBehavior(float range, float speed)
8         : hitRange(range), moveSpeed(speed) {}
9     // Basic linear movement implementation
10    virtual void move(Vec2& currentPos, const Vec2& targetPos) override {
11        Vec2 direction = (targetPos - currentPos).getNormalized();
12        currentPos += direction * moveSpeed;
13
14        // Basic hit check implementation
15        virtual bool checkHit(float distance) override {
16            return distance <= hitRange;
17        }
18        float getHitRange() const override { return hitRange; }
19        float getMoveSpeed() const override { return moveSpeed; }
20 };
21
22 // Sub attack behavior - Bridge Pattern's Concrete Implementor
23 // Hero-specific attack behaviors
24 class TaricBehavior : public BaseAttackBehavior {
25 public:
26     // Taric's unique attack properties
27     TaricBehavior() : BaseAttackBehavior(80.0f, 1.0f/6) {}
28
29     void move(Vec2& currentPos, const Vec2& targetPos) override {
30         // Taric's special movement pattern
31         BaseAttackBehavior::move(currentPos, targetPos);
32         // Add special effects...
33     }
34 };
35 // other heroes' attack behaviors...

```

3.3.4 Class diagrams



Left figure: Attack class diagram before refactoring



Right figure: Attack class diagram after refactoring with Bridge Pattern

Before Refactoring Class Diagram Explanation:

There's only one class that contains all the logic about attack. Attack behavior was hardcoded in Attack class and there's no way to customize attack patterns for different heroes. Movement and hit detection logic mixed together and it's difficult to modify attack behavior.

After Refactoring Class Diagram Explanation:

Bridge Pattern Implementation Structure changes attack behavior dynamically. There's clear separation between attack visuals and behavior. Besides, it's easy to add new attack patterns with better encapsulation of movement and hit detection logic.

1. IAttackBehavior (Implementor):

- Defines the interface for attack behaviors
- Abstracts movement and hit detection logic
- Provides methods for core attack functionality:
- move(): Handles movement patterns
- checkHit(): Determines hit conditions
- getHitRange(): Defines attack range
- getMoveSpeed(): Controls movement speed

2. Attack :

- Manages the visual representation and lifecycle
- Delegates behavioral decisions to IAttackBehavior
- Coordinates between game objects and attack behavior
- Handles timing and state management

3. Concrete Behaviors (Concrete Implementors):

- Special behaviors for unique hero attacks

4. Relationships:

- `Attack` holds a reference to `IAttackBehavior`
- Concrete behaviors implement `IAttackBehavior`
- `Attack` delegates movement and hit detection to behavior
- Behaviors can be swapped at runtime

3.3.5 The benefits

1. Separation of Concerns

Attack class focuses on lifecycle and coordination

Behaviors focus on movement and hit detection

Each class has a single, clear responsibility

2. Flexibility and Extensibility

New attack behaviors can be added without modifying Attack class

Behaviors can be changed at runtime

Easy to implement special attack patterns for different heroes

3. Better Maintainability

Changes to movement patterns don't affect core Attack logic

Hit detection can be modified independently

Clearer code organization and responsibility separation

4. Runtime Behavior Changes

Heroes can switch attack patterns dynamically

Different behaviors can be used based on conditions

Easier to implement power-ups or special states

3.4 Strategy Pattern (Behavioral Pattern)

3.4.1 Problem

In the original implementation, the Player class took on too much responsibility, handling multiple management logic related to the player, including management of gold, health, experience, heroes, and levels. This design leads to several core problems:

1. **Highly coupled code:** The Player class directly implements multiple management logic, including gold, health, experience, etc., resulting in tight coupling between different functions. Whenever a function changes, multiple parts of the code may need to be modified, increasing the maintenance complexity of the code.
2. **Unclear responsibility:** According to the Single Responsibility principle, a class should change for only one reason. However, the Player class takes on too many responsibilities, not only managing the player's behavior, but also managing multiple resources. Violation of SRP leads to unclear responsibilities of classes.
3. **Poor flexibility:** All the administrative logic is hard-coded in the Player class, any new functionality or requirements will have to change this class. The code for these classes is becoming more verbose and complex, with multiple administrative logics mixed together, making the code less readable and more difficult to maintain.
4. **Poor testability:** The management logic in the Player class is highly coupled, making testing difficult. For example, if you want to test your gold management logic, you might want to test the entire Player class. This increases the complexity of the unit test and reduces the independence and accuracy of the test.

3.4.2 Solution

In order to cope with the above problems, we adopt the strategy pattern. The strategy pattern is a behavioral design pattern that allows different behaviors or algorithms to be dynamically selected and switched at runtime. By extracting the different management logics, including heroes, ranks, coins, health, experience, etc, into separate strategy classes, the Player class can dynamically change behavior as needed without modifying the internal implementation.

Implementation steps:

1. Define multiple abstract classes to represent various management strategies, such as gold management, health management, experience management, hero management, level management, etc.
2. Create specific strategy classes for each management logic (e.g., StandardGoldStrategy, StandardBloodStrategy, etc.), and each strategy class implements the appropriate management behavior.

3. In the Player class, we use instances of these strategy classes to dynamically execute different management logic, and enable the Player class to switch strategies at runtime by means of dependency injection.

This way, each administrative logic is encapsulated in a separate strategy class, and the Player class only has to set different strategies to implement different behaviors. This not only decouples the complex logic of the Player class, but also improves the extensibility and maintainability of the code.

3.4.3 Code refactoring

The code we refactored is the player.cpp and Player.h files, which mainly include the Player class. In the refactored code, we split the Player class and extracted the tightly coupled management logic into multiple strategy classes in PlayerStrategy.cpp and PlayerStrategy.h. Here are some of the important refactoring changes:

1. Add strategy setting function

Here's the original Player.h file:



Before refactoring:

The code is messy and difficult to expand

```
1 class Player :public Sprite {
2 public:
3     // ...
4     void CreateHero();
5     void SetHero(Hero* hero);
6     void GetNetHero(NetHeroInfo* NetInfo);
7     bool addHeroToWar(Hero& hero);
8
9     bool hasEnoughGold(int amount);
10    void spendGold(int amount);
11    int getGold();
12    void addGold(int data);
13
14    void changeplayerBlood(int hurt);
15    int GetPlayerBlood();
16
17    int GetplayerExperience();
18    void SetplayerExperience(int x);
19    int GetplayerNeedExperience();
20    void SetplayerNeedExperience(int x);
21
```

```

22     int GetplayerLevel();
23     void SetplayerLevel(int x);
24     // ...
25 protected:
26     int gold;
27     // ...
28 };

```



After refactoring:

Add the strategy setting function, through the setGoldStrategy, setBloodStrategy and other functions, the Player class can dynamically set and change different strategies at runtime. Also add Strategy objects managed via smart pointers.

```

1 class Player :public Sprite{
2 public:
3     // Set different policies
4     void setGoldStrategy(std::shared_ptr<GoldStrategy> strategy);
5     void setBloodStrategy(std::shared_ptr<BloodStrategy> strategy);
6     void setExperienceStrategy(std::shared_ptr<ExperienceStrategy> strategy);
7     void setHeroStrategy(std::shared_ptr<HeroStrategy> strategy);
8     void setLevelStrategy(std::shared_ptr<LevelStrategy> strategy);
9 protected:
10    int gold;
11    // ...
12 private:
13    // Strategy objects managed via smart pointers
14    std::shared_ptr<GoldStrategy> goldStrategy;
15    //...other strategy objects
16 };

```

2. Implementation of the separation strategy



After refactoring:

Each strategy class is responsible for its own business logic, and interacts with the Player class through the interface, which makes each strategy can be extended and modified independently without affecting the implementation of the Player class.

The interface is split in PlayStrategy.h, and the implementation of the strategy class is in PlayerStrategy.cpp.

```

1  class HeroStrategy {
2  public:
3      virtual void createHeroes(Player* player) = 0;
4      virtual void setHero(Player* player, Hero* hero) = 0;
5      virtual bool addHeroToWar(Player* player, Hero& hero) = 0;
6      virtual void GetNetHero(Player* player, NetHeroInfo* NetInfo) = 0;
7      virtual ~HeroStrategy() = default;
8  };
9
10 class StandardHeroStrategy : public HeroStrategy {
11 public:
12     void createHeroes(Player* player) override;
13     void setHero(Player* player, Hero* hero) override;
14     bool addHeroToWar(Player* player, Hero& hero) override;
15     void GetNetHero(Player* player, NetHeroInfo* NetInfo) override;
16 };
17
18 class GoldStrategy {///...};
19 class StandardGoldStrategy : public GoldStrategy {///...};
20
21 class BloodStrategy {///...};
22 class StandardBloodStrategy : public BloodStrategy {///...};
23
24 class ExperienceStrategy {///...};
25 class StandardExperienceStrategy : public ExperienceStrategy {///...};
26
27 class LevelStrategy {///...};
28 class StandardLevelStrategy : public LevelStrategy {///...};

```

3. The Player class delegates to the strategy object



Before refactoring:

For example, when dealing with hero management, the Player class delegates heroStrategy to a concrete HeroStrategy implementation class to perform the addHeroToWar operation: This is the original addHeroToWar implementation.

```

1  bool Player::addHeroToWar(Hero& hero)
2  {
3      chessWarPlayer[heroNum].SetHeroDataByTag(hero.getTag(), hero.GetHeroPos());
4      chessWarPlayer[heroNum].setLevel(hero.getLevel());
5      chessWarPlayer[heroNum].SetOn(1);
6      heroNum++;
7      return true;

```



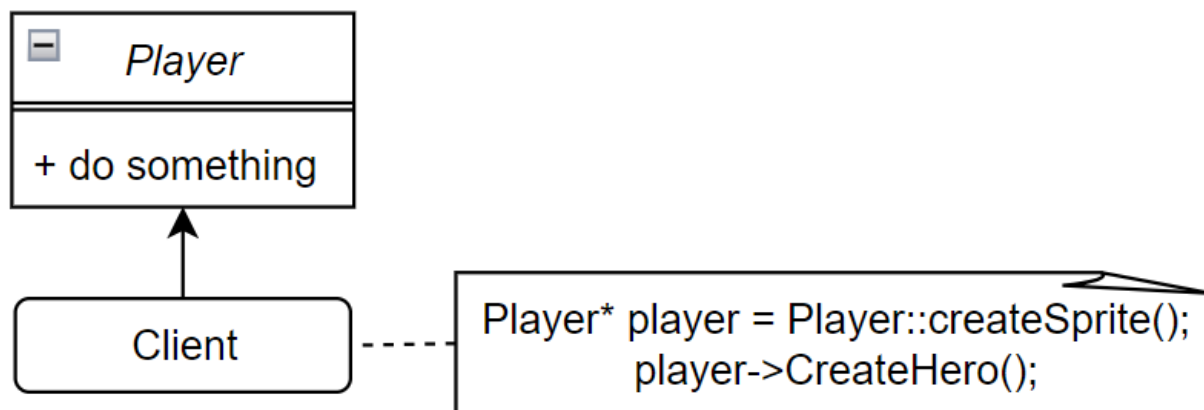
After refactoring:

This is the addHeroToWar function implemented using the strategy pattern. The addHeroToWar logic is no longer implemented directly by the Player class, but is instead delegated to heroStrategy, which gives us the flexibility to change the hero management strategy.

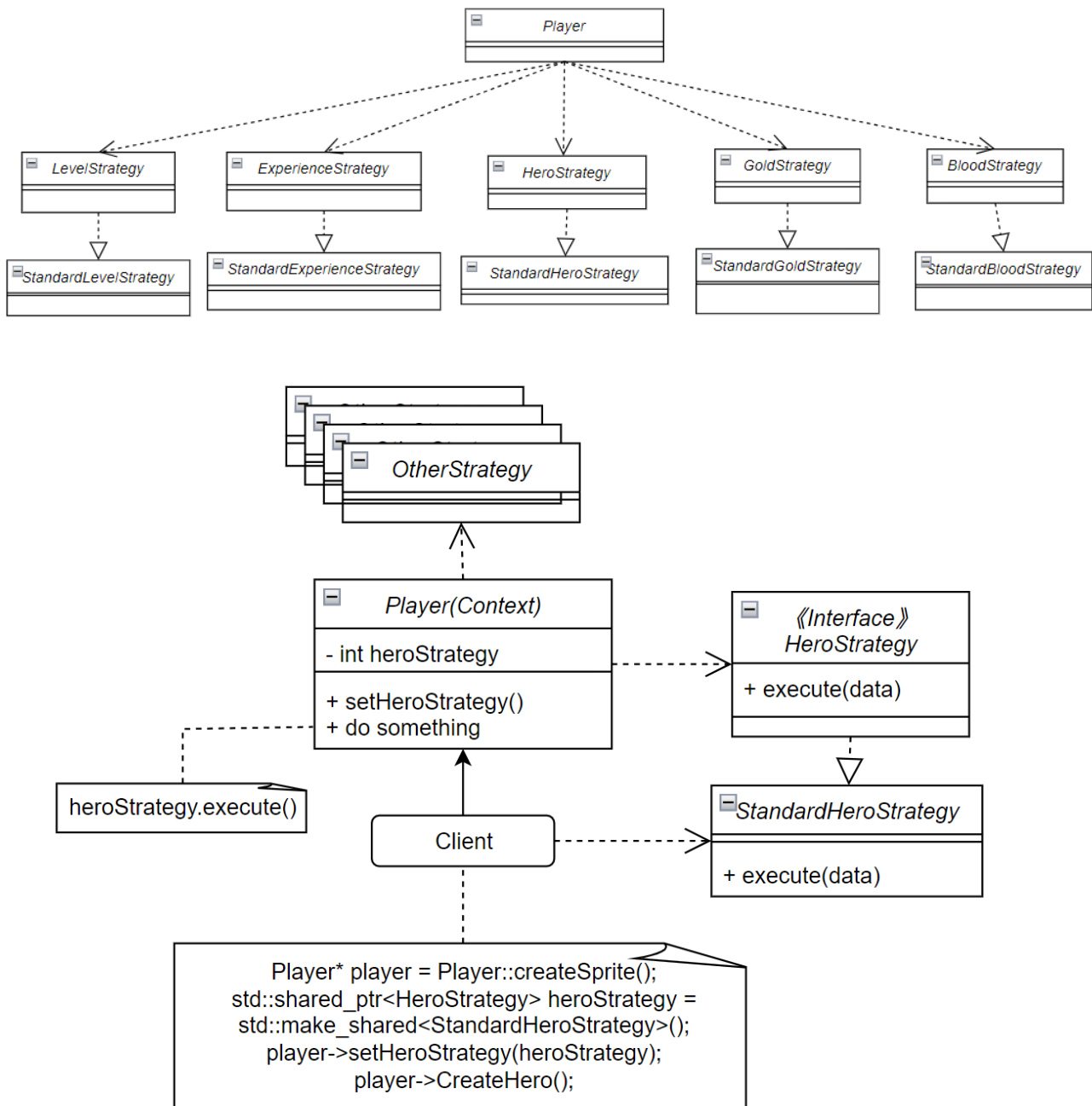
```
1 bool Player::addHeroToWar(Hero& hero) {
2     if (heroStrategy) {
3         return heroStrategy->addHeroToWar(this, hero);
4     }
5     return false;
6 }
```

3.4.4 Class diagrams

This is the Player class diagram before refactoring. Only the Player class is responsible for all player related operations:



Here's the class diagram for refactoring the Player class to use the strategy pattern:



In such a diagram, the main classes involved are Player and multiple strategy classes: HeroStrategy, GoldStrategy, BloodStrategy, ExperienceStrategy, LevelStrategy. Each strategy class has its own concrete implementation class, StandardHeroStrategy, StandardGoldStrategy respectively, StandardBloodStrategy, StandardExperienceStrategy, StandardLevelStrategy.

1. Class diagram Description:

- The Player class (context class) is the core class of the whole class diagram. As the context class, it holds references to all the strategy interfaces and calls the corresponding strategy methods according to the actual needs. The Player class has dependencies with the various strategy interfaces.

- HeroStrategy is an interface that defines abstract methods for different heroic actions. The HeroStrategy class provides Hero-related methods, which will be implemented by different strategy classes. The HeroStrategy interface is the core of the strategy pattern, which allows players to switch between different hero strategies as needed.
- StandardHeroStrategy is a concrete implementation of HeroStrategy that follows standard rules to create and manage heroes. It implements all of the methods defined by the HeroStrategy interface. Specifically, StandardHeroStrategy follows preset rules to create heroes, set hero properties, and add heroes to battle.
- The client is responsible for initializing and controlling the behavior of the Player object. The client will set the Player's strategy according to different requirements, for example, it can set different hero strategies for the Player.
- The OtherStrategy class contains all other strategies. These strategies will be selected and switched by the Player object as needed. The different implementation classes of the strategy will define exactly how to handle the corresponding game logic.

2. All strategy interface:

- HeroStrategy: This defines the strategy actions associated with heroes.
- GoldStrategy: This defines the strategy actions associated with gold coins.
- BloodStrategy: This defines the strategy operations related to the amount of blood.
- ExperienceStrategy: This defines the strategy actions associated with experience values.
- LevelStrategy: This defines the level-related strategy operations

3. All concrete strategy implementation classes:

- StandardHeroStrategy: This implements the HeroStrategy interface.
- StandardGoldStrategy: Implements the GoldStrategy interface.
- StandardBloodStrategy: Implements the BloodStrategy interface.
- StandardExperienceStrategy: ExperienceStrategy interface is realized.
- StandardLevelStrategy: This implements the LevelStrategy interface.

4. Relationship between classes:

There are dependencies between the Player class and each strategy interface, indicating that the Player can choose different strategies to manage and handle different resources and behaviors. Each concrete strategy class implements the corresponding strategy interface and provides a concrete implementation.

3.4.5 The benefits

1. **Decoupling:** The strategy pattern extracts different management logic into separate strategy classes, decoupling the Player class from these management logic. The Player class only interacts with the concrete strategy through the strategy interface and does not need to care about the concrete implementation of the strategy.
 2. **Enhanced flexibility:** Since the strategy pattern allows for dynamic selection of strategies at runtime, the Player class can flexibly switch behaviors according to different requirements.
 3. **Comply with the Open/Closed principle:** It means that the functionality can be extended by adding new strategy classes without modifying existing code. The new strategy class can provide a new implementation without affecting the existing functionality.
 4. **Improve testability:** The strategy pattern encapsulates each management logic in a separate strategy class, so that each strategy class can be tested independently. When testing, we only need to focus on the behavior of a single strategy, and do not need to consider other complex functions in the Player class.
 5. **Concise and Maintainable:** Each strategy class is responsible for only one function, so its responsibility is clear and easy to maintain.
-

3.5 State Pattern (Behavioral Pattern)

3.5.1 Problem

In the original code, the Hero class is both responsible for the state management of the hero and directly implements the specific logic of various states (e.g. attack, move, idle, recover, die, etc.). Although this design is intuitive, with the expansion of the game's functions, the following core problems were gradually exposed:

1. **Over-centralized responsibilities:** The Hero class needs to both determine the state and implement the specific behavior for each state. For example, the Hero::update function mixes the logic of multiple states (e.g. attack logic, movement logic, etc.), which is a huge and confusing amount of code. Violates the Single Responsibility Principle (SRP) in that the Hero class must be modified when a state is added or modified, increasing maintenance complexity.
2. **High code coupling:** The logic coupling between states is serious. Hero class needs to check the flag bits (such as isdead, ismove, isattacking, etc.) frequently, which makes it difficult to be independent between states. Conditions between different states may conflict, making it difficult to identify the source of the problem during debugging.

3. **Poor scalability:** Adding new states requires modifying logic in several places, such as `Hero::update` and other behavioral functions. As the number of states increases, the number of if-else or switch-case judgments increases, making the code unintuitive and difficult to maintain.
4. **Difficult to switch state dynamically:** State switching is realized by conditional branching, which makes it difficult to dynamically adjust the state logic. For example, if an attack requires an emergency switch to the dead state, the logic may become complex and confusing.
5. **Difficult to test:** Since the `Hero` class is directly responsible for state management and logic implementation, testing a state requires loading other logic of the entire class. It is difficult to independently cover the behavior of each state in the unit test, which may cause hidden problems.

3.5.2 Solution

To cope with the above problems, we introduce the state pattern to optimize the code design. By separating the logic of each state into a separate state class, the state pattern allows the behavior of the object to change dynamically with the state change. This simplifies the design of the `Hero` class so that it focuses on managing state switching and no longer implements state logic directly.

1. Define the state interface

Create a `HeroState` abstract class that defines the public interface for all states, including:

- `enter(Hero* hero)`: the logic to execute when entering a state.
- `update(Hero* hero, float dt)`: the logic to be executed at each frame of the state.
- `exit(Hero* hero)`: the logic to clean up when leaving the state.

2. Encapsulate each state into a separate class

Create specific classes for each state, such as `IdleState`, `AttackState`, `MoveState`, `RecoverState`, `DeadState`.

Each state class implements its own `enter`, `update` and `exit` methods.

3. Introduce state management in Hero class

Add a pointer to current state `currentState` in `Hero` class. Provide `changeState(HeroState* newState)` method which is responsible for state switching and calling state class methods.

Through this design, the `Hero` class only needs to manage the state switching, and the concrete logic is realized by the state class independently, which reduces the coupling and improves the scalability and maintainability.

3.5.3 Code refactoring

1. Creating State Interfaces



Before refactoring:

State switching and judgment rely on a large number of bool variables (e.g. isOn, isDead, isRecover...), with decentralized judgment logic

```
1 class Hero : public Sprite{
2 ...
3 private:
4     bool ison;//whether or not to play
5     bool isdead;//whether dead or not
6     bool isatk;//whether is attacking
7     bool ismove;//whether is moving
8     bool isrecover;//whether or not to recover blood for teammates
9     ...
10 }
```



After refactoring:

State switching is encapsulated in state classes, removing bool variables and centralizing logic for cleaner and more maintainable code.

```
1 //HeroState.h
2 class HeroState {
3 public:
4     virtual ~HeroState() = default;
5     virtual void enter(Hero* hero) = 0;          // Logic when entering a state
6     virtual void update(Hero* hero, float dt) = 0; // update every frame
7     virtual void exit(Hero* hero) = 0;          // Logic when leaving state
8 };
```

```
1 //HeroState.cpp
2 // -----Idle -----
3 void IdleState::enter(Hero* hero) {
4     CLOG("Hero enters Idle state");
5     hero->stopAllActions();
6 }
7
8 void IdleState::update(Hero* hero, float dt) {
```

```

9  //Idle logic
10 }
11
12 void IdleState::exit(Hero* hero) {
13     CCLOG("Hero exits Idle state");
14 }
15
16 // -----Move -----
17 void MoveState::enter(Hero* hero) {
18     CCLOG("Hero enters Move state");
19     ...
20 }
21
22 void MoveState::update(Hero* hero, float dt) {
23     //move logic
24 }
25
26 void MoveState::exit(Hero* hero) {
27     CCLOG("Hero exits Move state");
28     hero->stopAllActions();
29 }
30
31 // -----Attack -----
32 void AttackState::enter(Hero* hero) {
33     CCLOG("Hero enters Attack state");
34     ...
35 }
36
37 void AttackState::update(Hero* hero, float dt) {
38     //attack logic
39 }
40
41 void AttackState::exit(Hero* hero) {
42     CCLOG("Hero exits Attack state");
43     hero->stopAllActions();
44 }
45
46 // -----Recover -----
47 void RecoverState::enter(Hero* hero) {
48     CCLOG("Hero enters Recover state");
49     ...
50 }
51
52 void RecoverState::update(Hero* hero, float dt) {
53     hero->changeState(new IdleState());
54 }
55

```

```

56 void RecoverState::exit(Hero* hero) {
57     CLOG("Hero exits Recover state");
58 }
59
60 // -----Dead -----
61 void DeadState::enter(Hero* hero) {
62     CLOG("Hero enters Dead state");
63     ...
64 }
65
66 void DeadState::update(Hero* hero, float dt) {
67     // no need to update in dead state
68 }
69
70 void DeadState::exit(Hero* hero) {
71     CLOG("Hero exits Dead state");
72 }
73

```

2. Introducing state switching in the Hero class



Before refactoring:

The update function contains all the state logic and uses conditional judgment for state management

```

1 void Hero::update(float dt)
2 {
3     if (heroHP->getCurrentProgress() <= 0)
4     {
5         this->isdead = 1; //dead
6         this->setVisible(0);
7         heroHP->setVisible(0);
8         heroMP->setVisible(0);
9     }
10
11     if (isdead) //if dead , stop attacking
12     {
13         isatk = 0;
14         //heroAttack->setVisible(0);
15         this->stopAllActions();
16         this->unscheduleUpdate();
17         //this->Destory();
18     }

```

```

19
20     if (!ismove&&isdead==0)
21     {
22         //target in scope, attack
23         if (CountDistance() <= baseInfo->atkrange)
24         {
25             if (curAtkTimer == atktimer && atkTarget->IsDead() == 0) { //time
to attack
26                 isatk = 1;
27                 HeroAttack();
28                 ChangeMP();
29             }
30             curAtkTimer++;
31             if (curAtkTimer > atktimer)
32                 curAtkTimer = 0;
33         }
34
35         //target not in scope
36         else
37         {
38             this->stopAllActions();
39             ismove = 1;
40             HeroMoveTo();
41             movetimer = movetime * 60;
42             curMoveTimer = 0;
43         }
44     }
45     if (ismove&&isdead==0) {
46         curMoveTimer++;
47         if (curMoveTimer >= movetimer || CountDistance() <= baseInfo-
>atkrange) {
48             ismove = 0;
49             atktimer = baseInfo->atkspeed * 60;
50             curAtkTimer = atktimer - 20;
51         }
52
53     }
54     isupdate = 1;
55 }

```

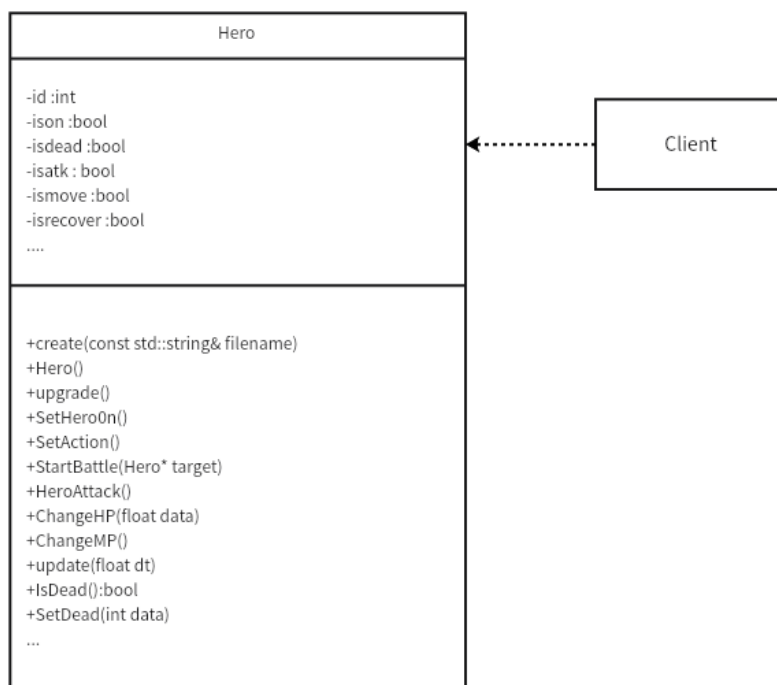
😊 After refactoring:

Only the update method of the current state is called (e.g. currentState->update(this, dt)). Specific logic is decentralized to each state class (e.g. IdleState,

AttackState, etc.), and the responsibilities between states are clear and easy to extend and maintain.

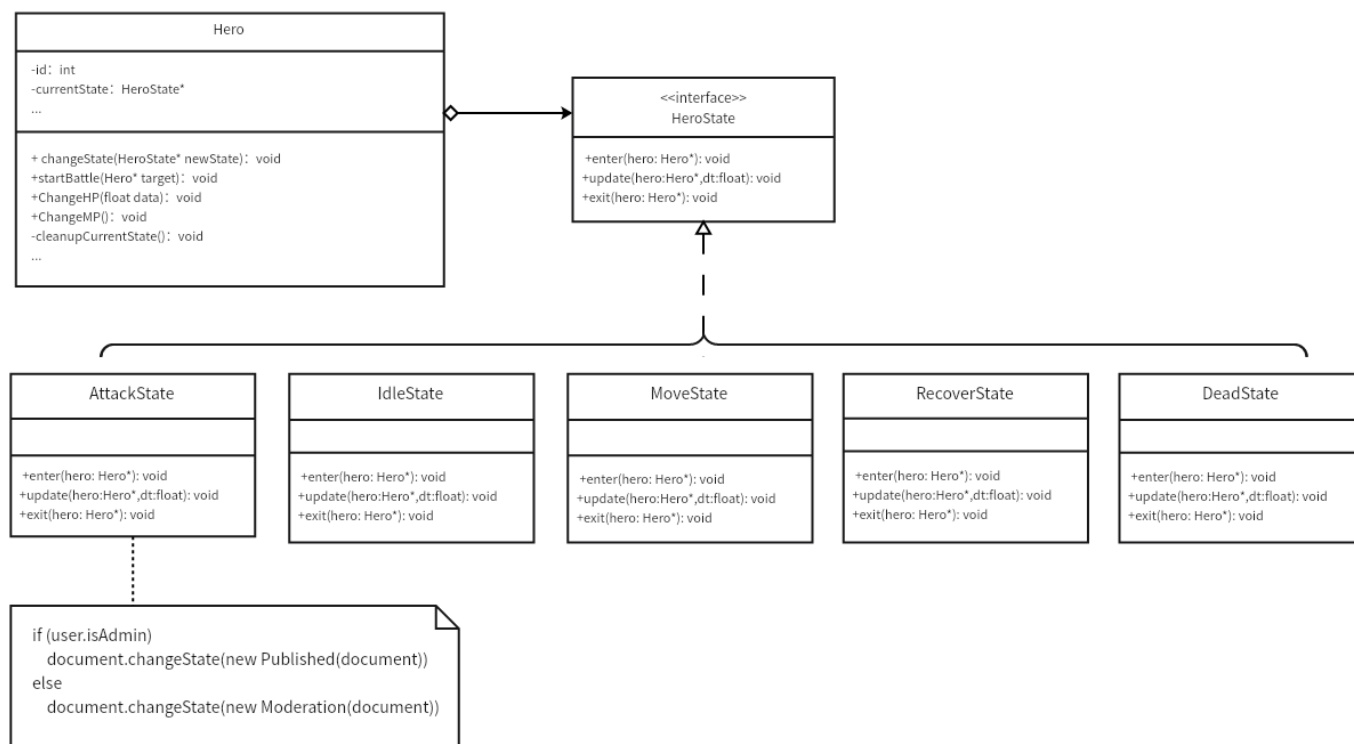
```
1 void Hero::update(float dt) {
2     if (currentState) {
3         currentState->update(this, dt);
4     }
5 }
6 void Hero::changeState(HeroState* newState) {
7     if (currentState) {
8         currentState->exit(this);
9         cleanupCurrentState();
10    }
11    currentState = newState;
12    if (currentState) {
13        currentState->enter(this);
14    }
15 }
```

3.5.4 Class diagrams



Before refactoring, the Hero class took on almost all of the functionality associated with heroes. It contains basic attributes (such as id, ison, etc.), and is also directly responsible for combat logic, state judgment, and even animation logic. All the state switching logic is recorded and controlled by boolean variables (e.g. isdead, isatk, etc.), and the logic between states is completely mixed together.

The Hero class is overly bloated, and all the functions are written together, which makes the class very complex, and if there is a slight need for modification, you will have to dive into the huge code to find the relevant logic. When adding new functionality (e.g., adding a new state), this design requires large-scale changes to existing code, which is difficult to maintain.



- Hero class: context class, holds the current state `currentState`, responsible for switching state and calling behavioral methods of current state.
- HeroState interface: state abstraction, defines general methods (`enter`, `update`, `exit`) for specific state classes to realize.
- concrete state classes: e.g. `IdleState`, `AttackState`, each class implements its own behavior logic (e.g. `move`, `attack`, `recover`).

After the refactoring, the responsibilities of Hero class have been cut down drastically, it only retains the content related to the core data of the hero, such as `id` and current state `currentState`, and adds a method `changeState()` to realize the state switching. The original complex state logic has been separated into the `HeroState` interface and its five subclasses (`IdleState`, `MoveState`, `AttackState`, `RecoverState` and `DeadState`). Each state subclass focuses on its own logic, e.g. `AttackState` only cares about the behavior of the attack.

Now, the Hero class doesn't need to know the state-specific logic, it just calls methods via the current state object `currentState`, e.g. `currentState->update(this, dt)`. The switching between states is realized by `changeState()`, and all the concrete behavior logic is done by the state class itself.

3.5.5 The benefits

1. **Decoupling:**The state logic is separated into independent state classes, and the Hero class no longer directly implements state-specific logic.The state classes are not dependent on each other, which reduces the coupling of the code.
2. **Single responsibility:**Each state class is only responsible for managing its own state logic, which conforms to the single responsibility principle (SRP).The Hero class is only responsible for managing state switching.
3. **Stronger extensibility:**When adding a new state, you only need to add a new state class without modifying the existing code.Conforms to the Open-Closed Principle (OCP), which improves the scalability of the code.
4. **Flexible dynamic behavior:**Support dynamic state switching, for example, adjusting the behavior of Hero in real time according to the game logic.It is more suitable for realizing complex game logic (e.g. AI, skill release, etc.).

With the introduction of state mode, the design of Hero class is more in line with object-oriented design principles, and the maintainability, extensibility and testing efficiency of the code have been significantly improved.

3.6 LazyLoading (Additional Pattern)

3.6.1 Problem

Lazy loading in `Attack.cpp` can improve performance by deferring the initialization of heroes and targets until they are needed. However, it introduces certain challenges:

1. **Unnecessary Initialization of All Heroes:** The current implementation initializes all heroes at the start of the battle, even if only a subset is involved in the battle. This is inefficient as it consumes memory and processing time for heroes that are not actively participating.
2. **Unnecessary target updates:** Every time an attack happens, the system checks and possibly updates all targets, even if some targets have already been set or don't need to be updated. This can result in making the process inefficient.
3. **Re-initialization of Dead Heroes: Every** time a hero dies, it is likely re-initialization or reset. This can lead to unnecessary memory operations, especially in a game with many heroes.

3.6.2 Solution

1. **Lazy Load Heroes:** Instead of initializing all heroes at the start, initialize them only when they are required (e.g., when they enter the battle or when their presence is actually needed).
2. **Update Targets Only When Necessary:** Instead of always checking and re-initializing all targets, only load the targets when they are about to be attacked or are about to change their state (like death or new target acquisition).

3. **Reset Only When Needed:** Instead of reloading or initializing all heroes upon reset, only reset heroes that need to be revived. If a hero is dead and required to be reloaded, do it on demand.

3.6.3 Code refactoring

1. SetHeroData() in Battle.cpp



Before: Go through all heroes, which can lead to performance issues.

```
1 void Battle::SetHeroData() {
2     // Extract hero information from the player
3     myHeroNum = myPlayer->GetHeroNum();
4     enHeroNum = enPlayer->GetHeroNum();
5
6     // Load player's heroes with lazy loading
7     for (int i = 0; i < myHeroNum; i++) {
8         if (!self[i]) { // Lazy loading if the hero hasn't been created yet
9             self[i] = Hero::create();
10            this->addChild(self[i], 2);
11            self[i]->HPInit();
12            self[i]->SetHeroDataByTag(myPlayer->GetWarHero()[i].getTag(),
myPlayer->GetWarHero()[i].GetHeroPos());
13            self[i]->setLevel(myPlayer->GetWarHero()[i].getLevel());
14            self[i]->updateInfo();
15            self[i]->SetDead(0);
16            self[i]->setVisible(1);
17            self[i]->SetHeroOn();
18            self[i]->setScale(0.45);
19            self[i]->retain(); // Retain to prevent memory issues
20        }
21    }
```



After: Only when the hero is in my team, improved performance.

```
1 void Battle::SetHeroData() {
2     // Extract hero information from the player
3     myHeroNum = myPlayer->GetHeroNum();
4     enHeroNum = enPlayer->GetHeroNum();
5
6     // Load player's heroes with lazy loading
```

```

7     for (int i = 0; i < myHeroNum; i++) {
8         if (!self[i]) { // Lazy loading if the hero hasn't been created yet
9             self[i] = Hero::create();
10            this->addChild(self[i], 2);
11            self[i]->HPInit();
12            self[i]->SetHeroDataByTag(myPlayer->GetWarHero()[i].getTag(),
myPlayer->GetWarHero()[i].GetHeroPos());
13            self[i]->setLevel(myPlayer->GetWarHero()[i].getLevel());
14            self[i]->updateInfo();
15            self[i]->SetDead(0);
16            self[i]->setVisible(1);
17            self[i]->SetHeroOn();
18            self[i]->setScale(0.45);
19            self[i]->retain(); // Retain to prevent memory issues
20        }
21    }
22
23    // Load enemy heroes with lazy loading
24    for (int i = 0; i < enHeroNum; i++) {
25        if (!enemy[i]) { // Lazy loading if the hero hasn't been created yet
26            enemy[i] = Hero::create();
27            this->addChild(enemy[i], 1);
28            enemy[i]->HPInit();
29            enemy[i]->SetBaseInfo(enPlayer->GetWarHero()[i].GetBaseInfo(),
enPlayer->GetWarHero()[i].GetName(), enPlayer->GetWarHero()[i].GetHeroPos());
30            enemy[i]->setLevel(enPlayer->GetWarHero()[i].getLevel());
31            enemy[i]->updateInfo();
32            enemy[i]->SetDead(0);
33            enemy[i]->setVisible(1);
34            enemy[i]->SetHeroOn();
35            enemy[i]->setScale(0.45);
36            enemy[i]->retain(); // Retain to prevent memory issues
37        }
38    }
39 }

```

2. SetAtkTarget() in Battle.cpp



Before: Every time the SetAtkTarget function is called, it calculates the distance between each attacker (atk[i]) and every potential target (tar[j]) to find the closest one.

```

1 void Battle::SetAtkTarget(Hero** atk, Hero** tar, int atkNum, int tarNum) {
2     double minDistance = MaxDis;
3     Hero* atkTarget = nullptr;

```

```

4
5     for (int i = 0; i < atkNum; i++) {
6         if (atk[i] && !atk[i]->IsDead()) {
7             for (int j = 0; j < tarNum; j++) {
8                 if (tar[j] && !tar[j]->IsDead()) {
9                     double temp = CountDistance(atk[i]->getPosition(), tar[j]-
>getPosition());
10                    if (temp <= minDistance) {
11                        minDistance = temp;
12                        atkTarget = tar[j];
13                    }
14                }
15            }
16            if (atkTarget) {
17                atk[i]->StartBattle(atkTarget); //Attack Target On Demand
18            }
19        }
20    }
21 }

```



After: The most significant change is the use of lazy loading for attack targets. Targets are only assigned to attackers alive and do not have target. This avoids redundant target assignments.

```

1 void Battle::SetAtkTarget(Hero* atk, Hero* tar, int atkNum, int tarNum) {
2     Hero* atkTarget = nullptr; // Holds the current attack target
3     for (int i = 0; i < atkNum; i++) {
4         if (!atk[i]->IsDead()) { // Check if the attacker is alive
5             // Only search for a target if the attacker doesn't have one
6             if (!atk[i]->ShowTarget()) {
7                 atk[i]->SetTarget(Hero::create()); // Lazy load target object
8                 if needed
9             }
10            double minDistance = MaxDis; // Initialize the maximum distance
11            to compare with
12            for (int j = 0; j < tarNum; j++) {
13                if (!tar[j]->IsDead()) { // Only consider alive targets
14                    double temp = CountDistance(atk[i]->getPosition(), tar[j]-
>getPosition());
15                    if (temp <= minDistance) {
16                        minDistance = temp;
17                        atkTarget = tar[j]; // Update the nearest target

```

```

17         }
18     }
19 }
20     if (atkTarget) {
21         atk[i]->StartBattle(atkTarget); // Execute attack on the
target
22     }
23 }
24 }
25 }

```

3. ResetHero() in Battle.cpp

☹ Before: All heroes are reset regardless of state.

```

1 void Battle::ResetHero() {
2     isNext = 0;
3     for (int i = 0; i < myHeroNum; i++) {//If the hero dies, then lazy load
4         if (self[i]) {
5             self[i]->SetDead(0);
6             self[i]->SetOn(1);
7         }
8     }
9     for (int i = 0; i < enHeroNum; i++) {//If the hero dies, then lazy load
10        if (enemy[i]) {
11            enemy[i]->SetDead(0);
12            enemy[i]->SetOn(1);
13        }
14    }
15 }

```

😊 After: Only reset dead heroes (avoids unnecessary resets for living heroes).

```

1 void Battle::ResetHero() {
2     isNext = 0;
3     for (int i = 0; i < myHeroNum; i++) {//If the hero dies, then lazy load
4         if (self[i]) {
5             self[i]->SetDead(0);
6             self[i]->SetOn(1);
7         }

```

```

8      }
9      for (int i = 0; i < enHeroNum; i++) {//If the hero dies, then lazy load
10         if (enemy[i]) {
11             enemy[i]->SetDead(0);
12             enemy[i]->SetOn(1);
13         }
14     }
15 }

```

3.6.4 The benefits

1. **Reduced Memory Usage:** Objects are only loaded when needed, preventing unnecessary memory consumption. This is especially useful for large-scale applications, as it helps avoid wasting resources that aren't immediately required.
2. **Improved Performance:** By delaying the loading of objects, the initial loading time is reduced, leading to faster application startup and a more responsive system overall.
3. **Optimized Resource Management:** Lazy loading helps distribute the load of computation and resource consumption throughout the application's lifecycle, preventing high resource usage at startup and ensuring efficient utilization of resources.
4. **Enhanced User Experience:** Users can interact with the application faster, as resources are loaded in the background as needed. This reduces waiting times and improves responsiveness, leading to a smoother experience.

4. Other relevant or valuable information

4.1 Six refactoring approaches and design patterns we adopted

- **Factory Pattern :** Through the factory pattern, the object creation and use are decoupled, the object creation process is simplified, and the extensibility of the system is enhanced.
- **Facade patterns:** The unified interface provided by facade pattern simplifies the complexity of the system, makes it easier for external callers to interact with the system, and improves the ease of use of the code.
- **Bridge Pattern:** The bridge pattern is used to separate the abstraction and implementation, so that the implementation can be changed independently without modifying the client code, which improves the flexibility of the system.
- **Strategy Pattern:** By encapsulating the algorithm with the strategy pattern, the selection and use of the algorithm are separated, which facilitates the system to choose different behaviors dynamically at runtime and improves the flexibility of the system.

- **State patterns:** By encapsulating state-related behaviors into different state classes, it simplifies the processing of conditional statements and improves the maintainability and extensibility of state management.
- **LazyLoading Pattern:** By delaying object loading, unnecessary resource consumption and memory occupation are reduced, and system startup performance and resource management are optimized.

4.2 Refactoring benefits

- **Improve system flexibility and extensibility:** By introducing appropriate design patterns, the system becomes more modular and can easily cope with future requirements changes and functional extensions.
- **Simplifying code and improving readability:** Design patterns help us to separate and manage complex logic and repetitive code effectively, which improves the maintainability and understandability of code.
- **Enhance team collaboration and development productivity:** Formalized design patterns make it easier for team members to understand and modify each other's code, reducing communication and development costs.

4.3 Long-term impact on the project

- **Enabling fast iteration and feature extension:** The introduction of design patterns makes the development of new functions more efficient, the system can quickly adapt to new requirements and changes, and improve the development iteration speed.
- **Optimize system performance:** Lazy loading mode and state management optimize the use of resources and performance, reduce the resource consumption of the system, and improve operating efficiency.
- **Reduce technical debt:** Through normalization and refactoring, technical debt is effectively reduced, and the long-term maintenance cost of the system is also reduced.